

Furniguard: A MERN Stack-Based Platform for Personalized Interior Design Services

1st Faizan Shaikh

*Student, School of Engineering
P P Savani University
Surat, Gujarat, India
faizanshaikh9262@gmail.com*

2nd Akansha Saini

*Assistant Professor, School of engineering
P P Savani University
Surat, Gujarat, India
akansha.saini@pps.ac.in*

Abstract—The digitization of personalized services has driven significant growth in the online interior design market. This paper presents Furniguard, a comprehensive academic prototype of an online interior design marketplace built on the MERN stack (MongoDB, Express.js, React.js, Node.js) and architected using a modern microservices pattern. The platform connects users with professional designers through a curated catalog of over 100 styles organized into 10+ categories. Key functionalities are tailored to three distinct roles: users, designers, and administrators. The system features a seamless appointment booking module, secure payment processing via the Razorpay API, and an integrated real-time chat system powered by Socket.IO for direct communication. Security is a core component, with JWT-based authentication for session management and OTP verification for email and password resets. For the frontend, Furniguard leverages shadcn/ui for rapid development of a responsive user interface. This paper provides a detailed overview of the system's microservices architecture, a breakdown of its role-based functional modules, and a discussion of its security implementation and future potential. As a proof-of-concept, Furniguard demonstrates a robust and scalable model for building modern, feature-rich, service-oriented web applications.

Index Terms—Interior Design, MERN Stack, Razorpay, ShadCN, Web Application, E-commerce, System Architecture, Prototype

I. INTRODUCTION

The traditional approach to interior design often begins with finding a suitable designer and engaging in-person consultation. However, Most of the time identifying right designer who aligns consumer personal taste of design and interest is one of the most challenging task. Even if find a good designer consumer often struggle to arti their personalized interest and style. This gap often leads to consumer dissatisfaction and reduced motivation for designers.

In recent years, the industry of interior design has undergone a significant transformation and adopting modern technologies. While traditional methods persist, the interior design services are now accessible through multiple online platforms. This transformation is primarily driven by the consumer satisfaction, convenience and personalization provided by online platforms. Designers also gain opportunities to show their skill by providing personalized solutions to consumers.

Advancement in Modern web based technologies have enabled the seamless designer-client workflow, from consulta-

tion, selecting personalized styles, booking, payments to final implementation and solution.

Furniguard is an academic prototype developed to explore core technical and functional requirements of modern online interior design platforms. It is built on the MERN (MongoDB, Express, ReactJS, NodeJS) stack, one of the most popular frameworks in full stack development due to its use of JavaScript across the entire application layer. This simplifies development and ensures scalability for both frontend and backend operations.

The primary goal of Furniguard is to create a seamless bridge between consumer and designer by providing inspirational styles and designs, easy consultation and simplified bookings. Consumers can browse curated catalog of interior design styles, gain inspiration, and conveniently book consultations. Designers, on the other hand understand consumer preferences and deliver personalized design solutions. This platform aims to reduce communication gaps, enhance consumer satisfaction, and provide designers with an opportunity to showcase their skills and expertise.

The platform's core values :

- **Structured Categorization:** To simplify user navigation, the catalog of 100+ design styles is organized into 10 primary categories: Bathroom, Flooring, Furniture, Kitchen, Layout, Lighting, Living Room, Stairs, Textiles, and Wallpaper. This helps users to efficiently filter and discover personalized designs relevant to specific rooms or project needs.
- **Seamless Service Booking:** Users can explore and select a preferred style, browse available designers, and choose a consultation mode and book appointment with just a few clicks.
- **Secure and Simple Payments:** Payments are handled through the Razorpay gateway, which ensures that all transactions are secure, transparent, and straightforward for users. The platform collects predetermined percentage as commission from consultation fees, while remaining amount is transferred to the designer.
- **Real-Time Communication:** The platform uses Socket.IO to provides an integrated chat system that enables instant interaction between users and designers.

This eliminates communication delays in consultation, helps clarify requirements in real-time, and builds trust throughout the consultation process.

- **Administrative Oversight:** The admin panel provides centralized monitoring and management of all platform activities. The panel includes managing bookings, payments, and communications, with admin having secure access to user-designer chat logs for quality assurance and dispute resolution. Admins can also manage the catalog, and communicate directly with designers through a dedicated admin-designer chat.

A. Motivation

The motivation behind Furniguard is both personal and professional. It started with a passion for architecture and interior design, which gradually turned into a multi-year journey of building interior design websites — beginning with simple HTML pages and eventually growing into a full-stack application.

The current market is dominated by platforms like Houzz, Livspace, and HomeLane, which have made digitalized interior design services a huge success. However, these ecosystems are vast and tailored for large-scale projects, which can feel overwhelming for a user seeking simple, initial guidance. This revealed the need for an accessible and practical “first step” into professional interior design services.

Furniguard was designed to address this need by offering a straightforward process to browse styles, connect with designers, and book an affordable fixed-fee consultation. From an academic perspective, the project also serves as a focused exercise in prototype feasibility, using the MERN stack and Rapid Application Development (RAD) principles to effectively build targeted solutions. Overall, Furniguard reflects both passion and a practical, problem-solving approach to technology.

B. Contributions

This paper makes the following contributions to the domain of web application development and system design:

- 1) It presents a detailed architecture for a service-based marketplace using the MERN stack, clearly describing the interaction between frontend, backend, and databases.
- 2) It demonstrates the practical integration of a third-party payment gateway (Razorpay) within a Node.js environment, including order creation, payment processing, and verification workflows.
- 3) It illustrates the use of a modern UI component library (shadcn/ui) to accelerate the development of a professional, responsive, and user-friendly interface.
- 4) It integrates real-time communication using Socket.IO, enabling direct messaging between users and designers, as well as admin–designer interactions with read-only access for monitoring and quality assurance.
- 5) It provides a comprehensive description of the platform’s functional modules, including the style catalog, designer

booking system, payment processing, messaging module, and an administrative panel.

- 6) It includes illustrative mock diagrams, database schema examples, and sample code snippets to demonstrate key implementation details, serving as a reference for similar academic or prototype projects.

The remainder of this paper is organized as follows: Section II related work. Section III details the system architecture. Section IV describes the functional modules. Section V discusses the implementation details. Section VI outlines the user flow. Section VII addresses security. Section VIII discusses limitations, and Section IX suggests future work. Finally, Section X concludes the paper.

II. RELATED WORK

The online interior design and home improvement space has a variety of established platforms that provide useful context for Furniguard’s development. A brief overview of some key players:

- **Houzz:** A globally recognized platform, focusing on home creation and renovation. It offers a large-scale model connecting homeowners, designers, retailers, and vendors, combined with design photos and an e-commerce marketplace. The workflow follows: get inspiration, find a professional, select products, and finalize implementation.
- **Livespace:** Popular in the Indian market, this platform provides end-to-end interior design solutions, including initial consultation with a designer to get design ideas, book Livespace, finalize designs, manufacturing, and installation.
- **UrbanLadder:** Initially an e-commerce furniture store, UrbanLadder later expanded to provide design consultation services for customized furniture and kitchen modulars, boosting furniture sales and creating its own catalog.
- **IKEA:** Known for its global furniture production and retail presence, IKEA has introduced digital tools for planning room layouts and virtual consultation services. Customers purchase all required components for DIY layout installation. The platform offers dedicated expert-led sessions to plan complete layouts using IKEA store products and provides 3D visualization of the final layout.

While these commercial platforms are feature-rich and operate at scale, Furniguard is designed as a functional prototype with different objectives. The primary focus is not market capture but to serve as a proof-of-concept, demonstrating how a clean, modern technology stack can address specific user needs. The core loop of browsing, booking, and payment makes it a clear and reproducible example of a service-based marketplace. Key differentiators include:

- **Focused Consultation Model:** While large-scale platforms manage end-to-end design, manufacturing, and installation, Furniguard focuses on the initial consultation

phase, allowing users to explore styles and book designers with minimal commitment.

- **Accessible and User-Centric:** The platform targets first-time or small-scale interior design users, providing a low-risk entry point compared to vast ecosystems like Houzz or Livespace.
- **Structured, Modular Catalog:** Furniguard offers a clearly categorized style catalog with 10+ categories, making it easy for users to filter designs and providing a more navigable experience than large-scale platforms.
- **Real-Time Communication and Transparency:** Integrated chat with Socket.IO allows users and designers to interact instantly, while admin oversight ensures accountability — a level of transparency rarely highlighted in commercial platforms.
- **Proof-of-Concept for Prototype Use:** Furniguard provides a clear, reproducible example of a service-based interior design system for learning and experimentation rather than market capture or monetization.

In summary, Furniguard draws inspiration from existing platforms while providing a focused, easy-to-use prototype that demonstrates a modern approach to an online interior design platform.

III. SYSTEM ARCHITECTURE

Furniguard is built upon a classical three-tier architecture, a time-tested model for developing scalable and maintainable web applications. This separation of concerns allows for independent development and scaling of the different application layers: the presentation layer (frontend), the logic layer (backend), and the data layer (database).

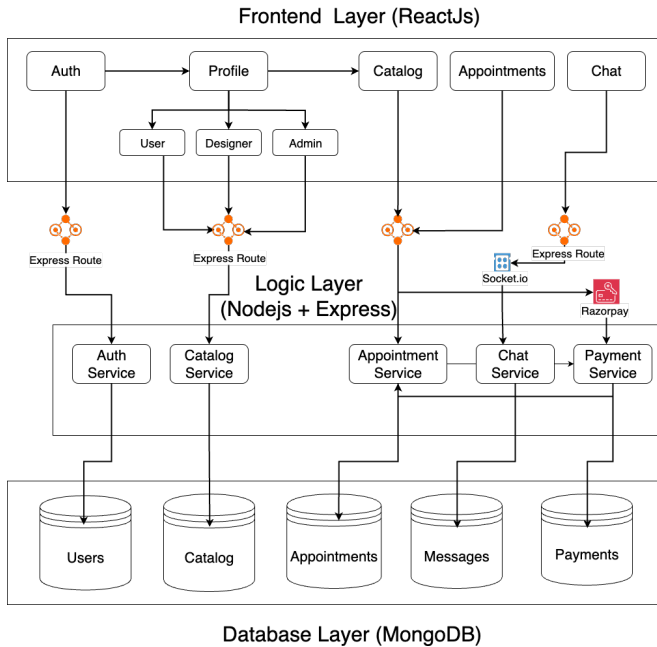


Fig. 1. Furniguard Three-Tier MERN Stack Architecture (Conceptual)

As depicted in Fig. 1, the components interact as follows:

- 1) **User, Designer, Admin** interacts with the application's user interface through a web browser on a client device, initiating all requests and actions.
- 2) The **Frontend**, a React-based Single Page Application (SPA), renders all UI components such as the Catalog, Styles, Appointments and Chat modules. It captures user inputs and interactions and sends corresponding API requests to the backend.
- 3) The **Backend** built with Node.js and Express.js, acts as the single entry point for all requests. It routes incoming traffic to the appropriate backend microservice (e.g. Auth Services, Appointmntnet Services, Catalog Services, Payment Services). Each service is responsible for a specific business logic, processing requests, and communicating with its dedicated database.
- 4) The **Database** layer uses MongoDB with Mongoose ODM(Object Document Mapping), with data organized into separate collections per service (e.g., Users, Payments, Messages, Appointments). This ensures data is managed independently by the service that owns it, which is a core principle of microservices design.

A. Frontend (Presentation Layer)

The frontend is built using React.js, a popular javascript library for modern Single Page Applications (SPAs), the User Interface is designed with **shadcn/ui** a modern components collection.

- **React Components:** The UI is broken down into small and reusable functional components with state managed by React Hooks ('useState', 'useEffect', 'useContext'). This modular approach simplifies state management, development and testing.
- **shadcn/ui:** A collection of reusable components built with Radix UI and Tailwind CSS. It allows for rapid development of a consistent, accessible, and highly customizable UI. Components like 'Card', 'Button', 'Dialog' (for modals), and 'Table' are used extensively.
- **API Communication:** The 'axios' library is used to make RESTful API calls to the backend server, handling requests-responses cycle in a clean, promise-based manner.

B. Backend (Logic Layer)

The backend is powered by the **Node.js** runtime environment, with the **Express.js** framework being the foundation for both the API Gateway and the individual microservice APIs.

- **API Gateway and Routing:** An Express.js server acts as the central API Gateway. Its primary role is to receive all incoming requests from the frontend and route them to the appropriate backend microservice. For example, a request to '/api/auth/login' is forwarded to the Auth Service, while a request to '/api/appointment' is routed to the Appointment Service.
- **Encapsulated Business Logic:** The core business logic is encapsulated within distinct microservices. Each service (e.g., Auth Service, Appointment Service, Chat Service,

Payments Service) has its own Express-based API for handling tasks specific to its domain, from validating user input to orchestrating complex workflows like payment processing with the Razorpay SDK.

- **Middleware:** Middleware functions are crucial for handling cross-cutting concerns. They are used for essential tasks such as parsing JSON request bodies ('express.json()'), enabling Cross-Origin Resource Sharing (CORS), and securing protected routes via JWT-based authentication and authorization.

C. Database (Data Layer)

MongoDB was chosen as the database due to its flexible, JSON-like document model, which aligns well with JavaScript-based backend logic.

- **Mongoose:** The Mongoose ODM (Object Document Mapping) is used to define schemas for the data models. Schemas provide a layer of structure and validation over the flexible MongoDB documents, reducing the chances of data-related errors.
- **Data Models:** Key collections of documents include 'Users', 'Designers', 'Styles', 'Payments', 'Messages' and 'Bookings'. The relationships between these collections are managed through references (storing ObjectIDs). Fig. 3 provides a conceptual overview of the schema.

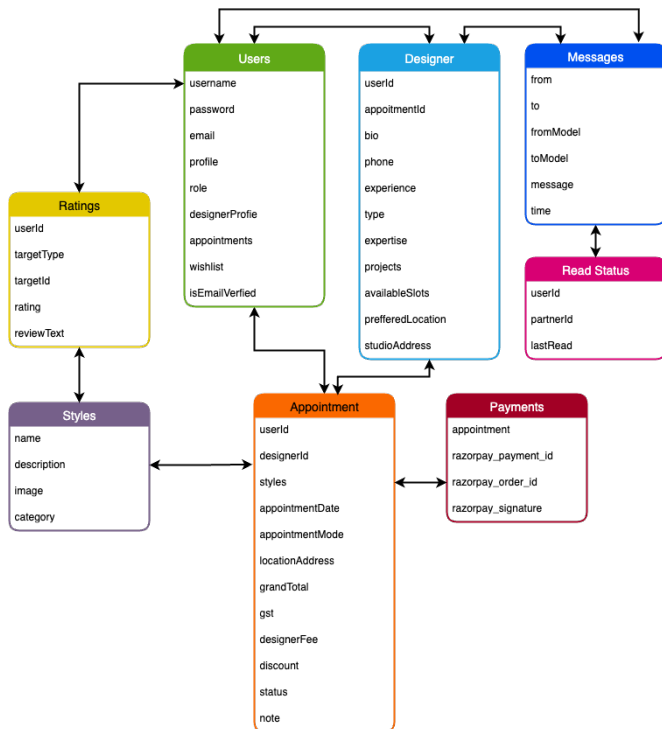


Fig. 3. Conceptual MongoDB Database Schema Overview

IV. FUNCTIONAL MODULES

The Furniguard platform is composed of several core modules, each responsible for a specific set of functionalities.

This section details the end-to-end flow, covering frontend interactions and based on that backend processes, and role-specific capabilities and permissions.

A. Style Catalog Browsing and Management

This module manages the creation, display, and browsing of interior design styles.

• Admin:

- Administrators can perform full CRUD (Create, Read, Update, Delete) operations on styles.
- Admins can add new styles by providing a name, description, category, and an image.
- Admins can update or remove styles as required.

• Frontend Display:

- All approved styles are displayed in the catalog using 'Card' components from shadcn/ui.
- The frontend uses a **masonry grid** layout for a visually appealing presentation of style images, names, and descriptions.

• User Interaction:

- Users can browse styles by category and view detailed style pages.
- Users can filter styles by category and search by name for quick style discovery.
- Users can add styles to their wishlist for later reference.
- Users can initiate bookings from the style details or dedicated appointments page for one or multiple styles.

• Designer Interaction:

- Designers can browse all styles to understand trends and improve consultation responses to users.
- Designers cannot book appointments themselves but can browse style catalog and use information to enhance recommendations.
- Designers have access to additional profile features such as expertise, projects, and biography to improve consultation quality.

• API Endpoints:

- POST /add – Admin-only endpoint to add a new style with image upload.
- PUT /update – Admin-only endpoint to update an existing style with optional image replacement.
- POST /remove – Admin-only endpoint to remove a style.
- GET /list – Admin-only endpoint to fetch all styles with full details.
- GET /user-list – Endpoint for users/designers to fetch the list of all approved styles.
- GET /home – Endpoint to fetch styles for home page display.
- GET /single – Admin-only endpoint to fetch details of a single style.

B. Designer Booking and Scheduling

This module enables users to book consultations with designers based on selected styles, and allows designers and admins to manage and monitor appointments.

- **Style Selection:**

- Users can select one or multiple styles from any category on the style details page.

- **Designer Selection:**

- Users choose a designer from the available list.

- **Appointment Form:**

- Users select a date from a calendar showing only the designer's available slots.
- Users can choose the appointment mode: Home, Studio, or Online.
- For Home or external locations, users can enter a custom address.
- Users may add personalized notes for the designer.
- The backend calculates the total cost including designer fee, GST, and any applicable discount.

- **Booking Submission:**

- Upon submission, a new appointment is created in the database with a `pending` status.
- Users are presented with a **Pay Now** option to complete the booking fee via Razorpay.

- **Payment Processing:**

- Payments are securely processed as described in the Payment Processing module.
- Upon successful payment, the appointment status is automatically updated to `confirmed`.

- **Appointment Management:**

- Users can cancel appointments before confirmation.
- Designers can view all appointments assigned to them and prepare for consultations.
- Admins can:
 - * View all user and designer appointments.
 - * Update appointment status to `completed` or `cancelled`.
 - * Delete appointments if necessary.

- **Data Structure:**

- Each appointment record stores the user, designer, selected styles, appointment date, mode, location, pricing details (grand total, GST, designer fee, discount), status, and optional notes.
- Timestamps track creation and updates for monitoring purposes.

- **API Endpoints:**

- `POST /new-booking` – Create a new appointment (user only).
- `GET /get-all` – Retrieve all appointments for the logged-in user.
- `GET /designer/get-all` – Retrieve all appointments assigned to the logged-in designer.

- `GET /admin/get-all` – Admin-only endpoint to fetch all user appointments.
- `POST /admin/update-status` – Admin-only endpoint to update appointment status.
- `POST /cancel` – User cancels an appointment.
- `POST /admin/delete` – Admin deletes an appointment.

C. Payment Processing with Razorpay

This module handles secure financial transactions between users and designers. It connects the booking process with confirmed payments and administrative monitoring.

- 1) **Order Creation (Backend):**

- When a user confirms their appointment and selects the **Pay Now** option, the frontend sends a request to the backend with the payment amount.
- The backend creates a Razorpay order, converting the amount from INR to paise (e.g., 500 INR → 50000 paise).
- The backend responds with the order details, including a unique `order_id`.

- 2) **Checkout (Frontend):**

- The frontend opens Razorpay's payment widget with the provided `order_id`.
- Users enter payment details and complete the transaction using available payment methods such as cards, UPI, or net banking.

- 3) **Payment Verification (Backend):**

- After payment, the frontend sends `razorpay_payment_id`, `razorpay_order_id`, `razorpay_signature`, and the `appointmentId` to the verification endpoint.
- The backend verifies the payment by recreating the signature and comparing it with the received `razorpay_signature`.
- If the signature matches, the payment is considered successful.

- 4) **Booking Confirmation:**

- Payment details are stored in the database and linked to the corresponding appointment.
- The appointment's status is updated from `pending` to `confirmed`.
- Users receive confirmation along with the payment reference.

- 5) **Payment Records and Admin Access:**

- All payment transactions can be retrieved through an admin-only endpoint.
- Records include linked appointment, user, designer, and selected style details.
- Payments are sorted by creation date for monitoring, reporting, and auditing purposes.

API Endpoints:

- `POST /checkout` – Create Razorpay order (requires user authentication).

- `POST /verification` – Verify payment and update booking status (requires user authentication).
- `GET /get-key` – Retrieve Razorpay API key for frontend integration (requires user authentication).
- `GET /all-payments` – Admin-only endpoint to fetch all payments with populated appointment, user, designer, and style details.

D. Admin Monitoring Dashboard

The administrative module provides comprehensive control over the platform.

- **Dashboard UI:** Private, password-protected section built with Table and Card components from shadcn/ui.
- **Booking Management:** Admin can view, update, cancel, or mark bookings as completed. Key metrics include total bookings, revenue, and designer activity.
- **Style Management:** Admin can add, update, or delete styles, including name, description, category, and image.
- **Designer Management:** Admin can manage designer profiles, including adding/removing access, viewing expertise, and handling chat permissions.
- **Chat Management:** Admin can view all chat logs in read-only mode. Admin can also communicate with designers.

E. Real-Time Communication with Socket.IO

This module provides an integrated, real-time chat system to facilitate instant and direct communication between platform participants, powered by **Socket.IO**. It is designed to support both consultation-related discussions and administrative support.

- **User-Designer Communication Channel:**
 - **Activation:** A private chat room is automatically created and becomes accessible to both the user and the designer once an appointment is booked.
 - **Functionality:** Users and designers can exchange text messages to discuss project requirements, clarify details before the meeting, and follow up after the consultation. This direct line of communication helps build trust and ensures both parties are aligned.
 - **Access:** Users and designers can access their chat history at any time from their respective Chats.
- **Admin-Designer Communication Channel:**
 - **Purpose:** A dedicated and separate communication channel exists between each designer and the platform administrators.
 - **Functionality:** This channel is used for official announcements, providing support to designers, resolving platform-related queries, and discussing payment or booking issues. This communication is not visible to standard users.
- **Administrative Oversight (Chat Log Monitoring):**
 - **Permissions:** Administrators have read-only access to all user-designer chat logs.

- **Purpose:** This capability is essential for quality assurance, moderating interactions, and for objective dispute resolution in case of a disagreement between a user and a designer. The read-only constraint ensures that admins cannot tamper with conversation records.

- **Technical Implementation (Socket.IO Events):**

- The chat system is event-driven. A persistent Web-Socket connection is established when a user enters a chat interface.
- **Room Management:** Private communication is managed using rooms. For each confirmed appointment, a unique room is created (e.g., named after the `appointmentId`).
- **Core Events:** Key events include `join_room` (when a user opens a chat), `send_message` (when a message is sent), and `receive_message` (when a message is broadcast to participants in the room).

- **API Endpoints:**

- `GET /inbox/user/:userId` – Retrieves the list of all conversations for the logged-in user.
- `GET /inbox/designer/:designerId` – Retrieves the list of all conversations for the logged-in designer.
- `GET /get-all` – An admin-only endpoint (secured by middleware) to fetch the complete list of all conversations on the platform for oversight.
- `GET /convo/:user1Id/:user2Id` – Fetches the complete message history for a specific conversation between any two parties (e.g., a user and a designer, or an admin and a designer).
- `POST /read-status` – Updates the read status of messages within a conversation to track seen/unseen messages.

F. Role-Based Access and Permissions

Furniguard supports three types of user roles with distinct functionalities:

- **User:** Can browse catalog, add styles to wishlist, book appointments (single/multiple styles), chat with designers.
- **Designer:** Can browse catalog, view appointments assigned to them, manage profile (projects, expertise, bio), and chat with users or admin. Cannot book appointments or manage wishlist.
- **Admin:** Full control over the platform, including adding/updating/deleting styles, managing bookings, handling designers, monitoring revenue, and accessing all chat logs in read-only mode.

V. USER FLOW AND DESIGN

The platform's user experience is designed around role-based workflows to provide an simple and easy interface for users, designers, and administrators. The overall application flow, from initial authentication to task completion, is tailored to each role's specific needs.

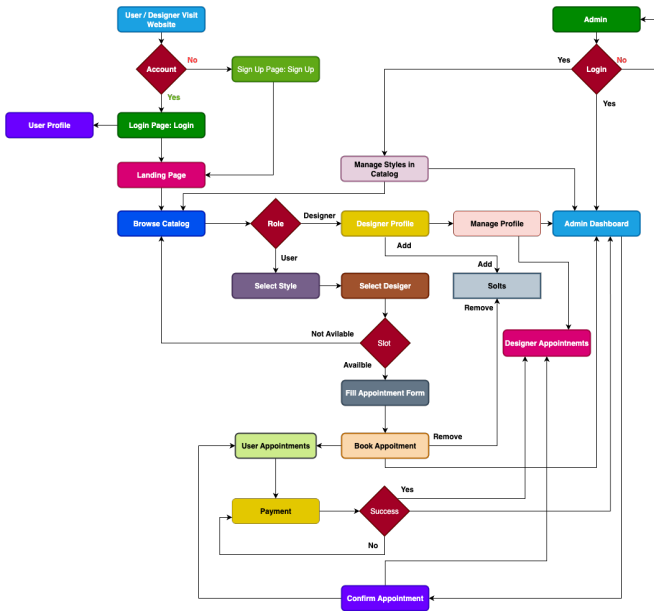


Fig. 4. Application Flowchart for Different User Roles

As illustrated in the flowchart (Fig. 4), the primary workflows are as follows:

- 1) **Authentication:** All users begin at the website's main entry point. A decision point checks if they have an existing account or not. New users are directed to a sign-up page, while existing users can log in with their existing account. A separate and secure login portal is provided for administrators.
- 2) **The Standard User's Booking Journey:** After logging in, a standard user follows these steps to book a consultation:
 - **Discovery and Selection:** The user is directed to the landing page, from where they can browse the catalog, select a preferred style, and then choose an available designer.
 - **Scheduling:** The user selects a specific time slot from the designer's availability and fills out the appointment form.
 - **Payment and Confirmation:** After reviewing the details, the user proceeds to payment. Upon a successful transaction via Razorpay, the appointment is confirmed and becomes visible in the "User Appointments" section of their profile.
- 3) **Designer's Workflow:** A logged-in designer is taken to their profile page. From here, they can manage their personal information, view their upcoming "Designer Appointments," and browse the style catalog.
- 4) **Administrator's Workflow:** An administrator logs in and is taken directly to the "Admin Dashboard." This central hub provides access to all management functions, including overseeing appointments, managing user and designer profiles, and maintaining the style catalog.

The UI design, facilitated by **shadcn/ui**, emphasizes clarity

and minimalism across all these workflows, using a consistent color palette and typography to create a professional and trustworthy feel.

VI. IMPLEMENTATION DETAILS

This section delves deeper into the technical implementation of the key components of the Furniguard platform.

A. Frontend Implementation

- **Project Setup:** The React application was initialized using Vite, a modern build tool that offers a faster development experience compared to Create React App.
- **State Management:** For a prototype of this scale, state is managed primarily through React's built-in hooks ('useState', 'useEffect'). For more complex state interactions, such as managing user authentication status, the Context API is used to avoid prop drilling.
- **Routing:** 'react-router-dom' is used to handle client-side routing, enabling navigation between different pages (e.g., '/', '/styles', '/booking') without full page reloads.
- **Styling:** Tailwind CSS is used for styling. This utility-first CSS framework, combined with shadcn/ui components, allows for building custom designs rapidly without writing custom CSS.

B. Backend Implementation

- **Project Structure:** The Node.js application follows a standard Model-View-Controller (MVC) pattern, with folders for 'routes', 'controllers', 'models', and 'middleware'. This separation keeps the codebase organized and scalable.
- **Environment Variables:** All sensitive information, such as database connection strings, Razorpay API keys, and JWT secrets, is stored in a '.env' file and loaded into the application using the 'dotenv' package. This is a crucial security practice to avoid hardcoding secrets in the source code.
- **Mongoose Schemas:** Detailed schemas are defined for each data model. For example, the 'Booking' schema includes fields for 'userId', 'designerId', 'styleId', 'appointmentDate', 'status' (enum: ['pending', 'confirmed', 'completed', 'cancelled']), and 'paymentDetails'.

C. Security Considerations

Security is a primary consideration in the design of the Furniguard platform, with several measures implemented to protect user data, ensure transactional integrity, and control access to the system.

- **Authentication and Access Control:** User identity and access are managed through a comprehensive, multi-layered authentication system:
 - **JWT-Based Sessions:** The platform uses JSON Web Tokens (JWT) for session management. Upon successful login, the server generates a signed JWT containing the user's role and ID. This token is sent

to the client and must be included in the ‘Authorization’ header of all subsequent requests to protected routes. A backend middleware is responsible for verifying the token’s signature and validity, ensuring that only authenticated users can access sensitive data or perform actions.

- **Email Verification via OTP:** To prevent the creation of accounts with fraudulent email addresses, new users must verify their email. Upon registration, an account is created in an “unverified” state, and a one-time password (OTP) is sent to the provided email. The user must enter this OTP to activate their account.
- **Secure Password Reset:** A “forgot password” feature is implemented using OTPs. A user can request a password reset, which triggers an email with a time-sensitive OTP. Only after successfully submitting this OTP is the user allowed to set a new password, ensuring that the request is legitimate.
- **Password Hashing:** All user passwords are never stored in plaintext. They are salted and hashed using a strong cryptographic algorithm like **bcrypt** before being stored in the database. This makes it computationally infeasible to reverse-engineer passwords even in the event of a data breach.
- **Payment Security:** Financial transactions are secured by delegating all payment processing to **Razorpay**, a PCI-DSS compliant third-party gateway. The Furniguard server never stores or handles raw credit card details. The security of the transaction is further enhanced by a **server-side verification** step, where a cryptographic signature generated by Razorpay is validated on the backend. This prevents client-side tampering and ensures that only legitimate, successful payments are recorded.
- **Infrastructure Security:** All sensitive credentials—including database connection strings, Razorpay API keys, and the JWT secret key—are stored securely in environment variables (‘.env’ file). These files are explicitly excluded from the version control system (e.g., via ‘.gitignore’) to prevent accidental exposure of secrets in the source code repository.

VII. DISCUSSION AND LIMITATIONS

As an academic prototype, Furniguard successfully demonstrates the architectural and functional requirements of a modern online service marketplace. The choice of the MERN stack proved highly effective for rapid, component-based development. Furthermore, the seamless integration of third-party services like Razorpay for payments and Socket.IO for real-time communication validates the feasibility of building robust, feature-rich web applications with this technology stack.

However, as a proof-of-concept, the project has several limitations that must be acknowledged:

- **Lack of Formal Evaluation:** The platform has not been subjected to formal, structured testing. This includes

usability testing with real end-users (both clients and designers), performance and load testing to identify bottlenecks, or a comprehensive security audit to uncover potential vulnerabilities. The feedback gathered to date has been informal and from a small peer group.

- **Scalability and Performance Not Tested:** The application currently runs in a development environment and has not been optimized for production-level traffic. A commercial deployment would require significant further work, including database indexing for efficient query performance, implementing caching strategies (e.g., with Redis) to reduce load, and deploying the microservices in a containerized environment (e.g., using Docker and Kubernetes) to enable horizontal scaling.
- **Simplified Business and Scheduling Logic:** While core features are in place, some of the more complex business logic has been simplified. For instance, the designer scheduling system does not yet handle intricate availability management, such as blocking off specific time slots, handling holidays, or managing recurring appointments. Additionally, while the system calculates commissions, it does not include an automated workflow for processing payouts to designers.

These limitations provide a clear roadmap for future development and underscore the project’s role as a foundational prototype rather than a market-ready product.

VIII. FUTURE WORK

Building on the current prototype, several enhancements can be implemented to move Furniguard closer to a production-ready application and a more innovative user experience.

- **Develop Designer Profiles and Dashboards:** A critical next step is to create a dedicated dashboard for designers. This would allow them to manage their profiles, set their availability using a dynamic calendar interface, track their bookings and earnings, and communicate more effectively with clients.
- **AI-Powered Design Generation:** To offer truly personalized options, an AI module could be developed. This feature would allow users to input parameters such as room dimensions, preferred color palettes, budget, and functional needs. The AI would then generate unique, customized interior design mockups, moving beyond the static catalog to offer limitless possibilities.
- **Immersive AR/VR Visualization:** To bridge the gap between imagination and reality, an Augmented Reality (AR) or Virtual Reality (VR) feature could be integrated. This would enable users to use their smartphone’s camera to visualize how different furniture items or entire design styles would look in their own physical space, significantly enhancing the decision-making process.
- **Advanced Recommendation Engine:** The existing user rating system can be leveraged to build a more sophisticated recommendation engine. Using machine learning models (such as collaborative filtering), the platform could suggest styles, furniture, and even designers to

users based on their browsing history, wishlist, and the ratings of other users with similar tastes.

- **Production Deployment and Scalability:** To handle real-world traffic, the application must be deployed to a cloud platform like AWS or Vercel. This involves setting up a production-grade database service (like MongoDB Atlas), containerizing the microservices using **Docker** for portability, and implementing a CI/CD (Continuous Integration/Continuous Deployment) pipeline for automated testing and seamless updates.

IX. CONCLUSION

Furniguard successfully demonstrates the design and implementation of an online interior design service platform using the MERN stack. It fulfills its objective as an academic prototype by integrating key functionalities such as a style catalog, an appointment booking system, secure payment processing with Razorpay, and an administrative monitoring dashboard. The use of modern tools like shadcn/ui allowed for the rapid development of a clean and responsive user interface, while the MERN stack provided a cohesive and efficient development environment.

While the current implementation has limitations, it provides a solid foundation for future academic exploration and development. The architecture is modular and extensible, allowing for the addition of more advanced features like user personalized recommendations, and AI powered style generation and Visualization. Ultimately, Furniguard serves as a valuable case study in building modern, full-stack web applications for the growing digital service economy.

ACKNOWLEDGMENT

I wish to extend my deepest appreciation to Akansha Saini, my faculty mentor, for their expert guidance and mentorship. Their insights were crucial in shaping the project's direction and technical architecture. Gratitude is also extended to my colleagues for their constructive feedback during the development stages.

REFERENCES

- [1] The React Team, "React Documentation," Facebook, Inc., 2025. [Online]. Available: <https://react.dev/>
- [2] OpenJS Foundation, "Node.js v20.11.0 Documentation," 2025. [Online]. Available: <https://nodejs.org/docs/latest-v20.x/api/>
- [3] MongooseJS, "Mongoose v8.1.1 Documentation," 2025. [Online]. Available: <https://mongoosejs.com/docs/guide.html>
- [4] Socket.IO, "Socket.IO Documentation," 2025. [Online]. Available: <https://socket.io/docs/v4/>
- [5] Auth0, "JSON Web Tokens Introduction," JWT.IO, 2025. [Online]. Available: <https://jwt.io/introduction>
- [6] Razorpay, "Razorpay Developer Documentation," Razorpay Software Private Limited, 2025. [Online]. Available: <https://razorpay.com/docs/>
- [7] Shadcn, "shadcn/ui - Beautifully designed components," 2025. [Online]. Available: <https://ui.shadcn.com/>
- [8] "Houzz - Home Design, Decorating and Remodeling Ideas," Houzz Inc., 2025. [Online]. Available: <https://www.houzz.com/>
- [9] "HomeLane - End-to-end Interior Design Solutions," HomeLane.com, 2025. [Online]. Available: <https://www.homelane.com/>